Abdullah Sonija 24k-0013

OS Assignment 2 question # 1

4.1 Provide three programming examples in which multithreading provides better performance than a single-threaded solution.

1. A web server that handles each client request in a separate thread, allowing simultaneous connections.
2. A word processor where one thread responds to user keystrokes while another thread performs background spell-checking or document saving.
3. An image rendering application where the image is divided into segments, and each segment is processed concurrently by different threads.

4.2 Using Amdahl's Law, calculate the speedup gain of an application that has a 60 percent parallel component for (a) two processing cores and (b) four processing cores.

Formula: Speedup = 1 / ((1 - P) + (P / N)) Where P = 0.60.

(a) For N = 2: Speedup = 1 / (0.4 + 0.6/2) = 1 / 0.70 = **1.42**

(b) For N = 4: Speedup = 1 / (0.4 + 0.6/4) = 1 / 0.55 = **1.81**

4.3 Does the multithreaded web server described in Section 4.1 exhibit task or data parallelism?

It exhibits **task parallelism**. Each new client request represents a unique task (serving a different user, potentially requesting different files or executing different scripts) running concurrently with others.

4.4 What are two differences between user-level threads and kernel-level threads? Under what circumstances is one type better than the other?

Differences:

- User-level threads are managed entirely by a user-space thread library without kernel knowledge, while kernel threads are managed directly by the OS.
- Context switching between user-level threads is faster because it doesn't require a system call (mode switch), unlike kernel threads.

Circumstances:

User-level threads are better when execution speed and low overhead are critical, and true multi-core parallelism isn't required. Kernel-level threads are better when applications

need to utilize true multiprocessor parallelism or when threads frequently perform blocking I/O (so one blocked thread doesn't block the whole process).

4.5 Describe the actions taken by a kernel to context-switch between kernel level threads.

The kernel must save the context (CPU registers, program counter, stack pointer) of the currently running thread into its Thread Control Block (TCB). It then updates the thread's state to "ready" or "blocked". The OS scheduler selects the next thread to run, and the kernel restores the context from the new thread's TCB into the CPU registers to resume its execution.

4.6 What resources are used when a thread is created? How do they differ from those used when a process is created?

Creating a thread requires allocating a Thread Control Block (TCB), a thread identifier, a program counter, a register set, and a stack. Process creation is much heavier, requiring memory allocation for code, data, and heap segments, file descriptors, and a full Process Control Block (PCB). Threads share the memory and resources of their parent process

4.7 Assume that an operating system maps user-level threads to the kernel using the many-to-many model and that the mapping is done through LWPs. Furthermore, the system allows developers to create real-time threads for use in real-time systems. Is it necessary to bind a real-time thread to an LWP? Explain.

Yes, it is necessary. Real-time threads require strict, guaranteed execution timelines. If it is not bound to a specific LWP (Lightweight Process), it might have to wait for an LWP to become available before it can be scheduled by the kernel. Binding it ensures the real-time thread always has an immediate kernel pathway to execution.

4.8 Provide two programming examples in which multithreading does not provide better performance than a single-threaded solution.

- A strictly sequential mathematical computation (like calculating the Fibonacci sequence) where step N strictly depends on step N-1.
- A simple shell script that executes a single, quick command. The overhead of creating and joining threads would take longer than the task itself.

4.9 Under what circumstances does a multithreaded solution using multiple kernel threads perform better than a single-threaded solution on a single-processor system?

When the application performs heavy or frequent **blocking I/O operations** (like reading from a disk or network). In a single-threaded process, the whole process blocks. With

multiple kernel threads, if one thread blocks for I/O, the kernel can context-switch to another thread that can continue utilizing the CPU.

4.10 Which of the following components of program state are shared across threads in a multithreaded process?

a. Register values
b. Heap memory
c. Global variables
d. Stack memory

b. Heap memory and **c. Global variables**. (Registers and stack memory are strictly per-thread).

4.11 Can a multithreaded solution using multiple user-level threads perform better on a multiprocessor system than on a single-processor system? Explain.

No. In a pure user-level thread model (many-to-one), the operating system kernel is completely unaware of the threads. It schedules the entire process onto a single processor core. Therefore, the user-level threads cannot execute in parallel on multiple cores.

4.12 In Chapter 3, we discussed Google's Chrome browser and its practice of opening each new tab in a separate process. Would the same benefits have been achieved if, instead, Chrome had been designed to open each new tab in a separate thread? Explain.

No. Threads share the same memory space. If Chrome used threads and one tab experienced a fatal error (like a segmentation fault) or suffered a security breach, it would crash the entire browser process or expose the data of all other tabs. Using separate processes ensures memory isolation and stability.

4.13 Is it possible to have concurrency but not parallelism? Explain.

Yes. Concurrency means that multiple tasks are making progress at the same time (e.g., through time-slicing on a single CPU core). Parallelism means tasks are executing exactly at the identical moment (requires multiple CPU cores). A single-core system multitasking is concurrent, but not parallel.

4.14 Using Amdahl's Law, calculate the speedup gain for the following applications:

- 40 percent parallel with (a) eight processing cores and (b) sixteen processing cores
- 67 percent parallel with (a) two processing cores and (b) four processing cores
- 90 percent parallel with (a) four processing cores and (b) eight processing cores

Formula: 1 / ((1 - P) + (P / N))

- **40% parallel (P=0.4):** (a) 8 cores: 1 / (0.6 + 0.4/8) = **1.53**. (b) 16 cores: 1 / (0.6 + 0.4/16) = **1.60**.
- **67% parallel (P=0.67):** (a) 2 cores: 1 / (0.33 + 0.67/2) = **1.50**. (b) 4 cores: 1 / (0.33 + 0.67/4) = **2.01**.
- **90% parallel (P=0.90):** (a) 4 cores: 1 / (0.1 + 0.9/4) = **3.07**. (b) 8 cores: 1 / (0.1 + 0.9/8) = **4.70**.

4.15 Determine if the following problems exhibit task or data parallelism:

- Using a separate thread to generate a thumbnail for each photo in a collection
- Transposing a matrix in parallel
- A networked application where one thread reads from the network and another writes to the network
- The fork-join array summation application
- The Grand Central Dispatch system

- Generating a thumbnail for each photo: **Data parallelism** (same operation on different data).
- Transposing a matrix: **Data parallelism**.
- Network read/write threads: **Task parallelism** (different operations/roles).
- Fork-join array summation: **Data parallelism**.
- Grand Central Dispatch: **Both** (it is a framework that schedules both data and task parallel workloads).

4.16 A system with two dual-core processors has four processors available for scheduling. A CPU-intensive application is running on this system. All input is performed at program start-up, when a single file must be opened. Similarly, all output is performed just before the program terminates, when the program results must be written to a single file. Between start-up and termination, the program is entirely CPU-bound. Your task is to improve the performance of this application by multithreading it. The application runs on a system that uses the one-to-one threading model (each user thread maps to a kernel thread).

- How many threads will you create to perform the input and output? Explain.
- How many threads will you create for the CPU-intensive portion of the application? Explain.

- **Input/Output: 1 thread**. Since all input/output goes to a single file, it is a strictly sequential bottleneck. Multiple threads trying to read/write a single file simultaneously would require locking, eliminating any parallel benefit.
- **CPU-intensive portion: 4 threads**. Since the system has two dual-core processors (4 available cores) and uses a one-to-one threading model, 4 threads will perfectly maximize CPU utilization without adding unnecessary context-switching overhead.

4.17 Consider the following code segment:

```
pid t pid;
pid = fork();
if (pid == 0) { /* child process */
fork();
thread create( . . .);
}
fork();
```

a. How many unique processes are created?
b. How many unique threads are created?

- **Processes:** 6 unique processes. (P0 forks P1. P1 forks P2. Then P0, P1, P2 all fork once, adding P3, P4, P5).
- **Threads:** 8 threads. All 6 processes have a main thread. Additionally, P1 and P2 each trigger the thread_create line explicitly creating 2 new threads. 6 + 2 = 8.

4.18 Linux does not distinguish between processes and threads. Instead, Linux treats both in the same way, allowing a task to be more akin to a process or a thread depending on the set of flags passed to the clone() system call. However, other operating systems, such as Windows, treat processes and threads differently. Typically, such systems use a notation in which the data structure for a process contains pointers to the separate threads belonging to the process. Contrast these two approaches for modeling processes and threads within the kernel.

Windows strictly differentiates between processes and threads; a process is a passive container of resources, and a thread is the active execution unit belonging to a process. Linux does not strictly differentiate them in the kernel; both are considered "tasks" and are created using the clone() system call. The degree of resource sharing (memory, files) is simply determined by the flags passed to clone().

4.19 The program shown below uses the Pthreads API. What would be the output from the program at LINE C and LINE P?

```
#include <pthread.h>
#include <stdio.h>
int value = 0;
void *runner(void *param); /* the thread */
int main(int argc, char *argv[])
{
pid t pid;
pthread t tid;
pthread attr t attr;
pid = fork();
if (pid == 0) { /* child process */
pthread attr init(&attr);
pthread create(&tid,&attr,runner,NULL);
pthread join(tid,NULL);
printf("CHILD: value = %d",value); /* LINE C */
}
else if (pid > 0) { /* parent process */
wait(NULL);
printf("PARENT: value = %d",value); /* LINE P */
}
}
void *runner(void *param) {
value = 5;
pthread exit(0);
}
```

- **LINE C (Child Process):** CHILD: value = 5. The child process creates a thread that updates the global variable value to 5.
- **LINE P (Parent Process):** PARENT: value = 0. Since fork() creates a completely separate memory space for the child process, the child's thread only modifies the child's copy of value. The parent's value remains unchanged.

4.22 Write a multithreaded program that calculates various statistical values for a list of numbers. This program will be passed a series of numbers on the command line and will then create three separate worker threads. One thread will determine the average of the numbers, the second will determine the maximum value, and the third will determine the minimum value.
For example, suppose your program is passed the integers 90 81 78 95 79 72 85
The program will report

The average value is 82
The minimum value is 72
The maximum value is 95

The variables representing the average, minimum, and maximum values will be stored globally. The worker threads will set these values, and the parent thread will output the values once the workers have exited. (We could expand this program by creating additional threads that determine other statistical values, such as median and standard deviation.)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int average = 0;
int maximum = 0;
int minimum = 0;

typedef struct {
    int *arr;
    int size;
} ThreadData;

void *calculate_average(void *param) {
    ThreadData *data = (ThreadData *)param;
    int sum = 0;
    for (int i = 0; i < data->size; i++) {
        sum += data->arr[i];
    }
    average = sum / data->size;
    pthread_exit(0);
}

void *calculate_maximum(void *param) {
    ThreadData *data = (ThreadData *)param;
    maximum = data->arr[0];
    for (int i = 1; i < data->size; i++) {
        if (data->arr[i] > maximum) {
            maximum = data->arr[i];
        }
    }
    pthread_exit(0);
}

void *calculate_minimum(void *param) {
    ThreadData *data = (ThreadData *)param;
    minimum = data->arr[0];
    for (int i = 1; i < data->size; i++) {
        if (data->arr[i] < minimum) {
            minimum = data->arr[i];
        }
    }
    pthread_exit(0);
}
```

```c
int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: ./stats <num1> <num2> ... <numN>\n");
        return -1;
    }
    int size = argc - 1;
    int *numbers = (int *)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        numbers[i] = atoi(argv[i + 1]);
    }
    pthread_t tid1, tid2, tid3;
    ThreadData data = {numbers, size};
    pthread_create(&tid1, NULL, calculate_average, &data);
    pthread_create(&tid2, NULL, calculate_maximum, &data);
    pthread_create(&tid3, NULL, calculate_minimum, &data);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);
    pthread_join(tid3, NULL);
    printf("The average value is %d\n", average);
    printf("The minimum value is %d\n", minimum);
    printf("The maximum value is %d\n", maximum);
    free(numbers);
    return 0;
}
```

4.23 Write a multithreaded program that outputs prime numbers. This program should work as follows: The user will run the program and will enter a number on the command line. The program will then create a separate thread that outputs all the prime numbers less than or equal to the number entered by the user.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *print_primes(void *param) {
    int max_num = atoi((char *)param);
    printf("Prime numbers up to %d:\n", max_num);
    for (int i = 2; i <= max_num; i++) {
        int is_prime = 1;
        for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                is_prime = 0;
                break;
            }
        }
```

```
        if (is_prime) {
            printf("%d ", i);
        }
    }
    printf("\n");
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: ./primes <number>\n");
        return -1;
    }
    if (atoi(argv[1]) < 2) {
        printf("Please enter a number >= 2\n");
        return -1;
    }
    pthread_t tid;
    pthread_attr_t attr;
    pthread_attr_init(&attr);
    pthread_create(&tid, &attr, print_primes, argv[1]);
    pthread_join(tid, NULL);
    return 0;
}
```